



Fig. 7: Types of harvesting

value-change dump (VCD) files are produced; one maximises power in all even cycles and the other maximises power in odd cycles.

3. Combine the even and odd power traces into a single peak power trace by taking power values from even cycles in the even trace, and odd cycles in the odd trace.

4. Peak power requirement of the application is the maximum power cycle value found in the peak power trace.

5. When making this calculation, for input-dependent branches the algorithm always take the most expensive one. For loops where the maximum number of iterations can be determined, simply take the energy for one iteration and multiply it by that maximum number. If neither is possible, it may not be possible to compute the peak energy of the application; however, this is uncommon in embedded applications.

6. **Results.** By accounting for all possible inputs using symbolic simulation, this technique can bound peak power and energy for all possible application executions without guard banding. Peak power requirements reported by the technique are 15 per cent lower than guard-banded application-specific requirements, 26 per cent lower than guard-banded stress-mark-based requirements, and 27 per cent lower than design-specification-based requirements. On an average, the peak energy requirements reported by the technique are 17 per cent lower than guard-banded applica-

tion-specific requirements, 26 per cent lower than guard-banded stress-mark-based requirements, and 47 per cent lower than design-specification-based requirements.

Optimising peak power guided by profile results

The technique can also be used to guide application-based optimisation by analysing the processor's behaviour during the cycles of peak power consumption. Three different optimisations can then be applied as appropriate:

1. Replace a complex instruction that induces a lot of activity in one cycle with a sequence of simpler instructions, thus spreading out the activity over several cycles

2. Delay the activation of one or more modules, previously activated in a peak cycle, until a later cycle

3. Avoid the multiplier being active simultaneously with the processor core by inserting a no-operation instruction (NOP) into the pipeline during the cycle in which the multiplier is active

Taking combined figures across the benchmarks, these techniques reduced peak power by up to 10 per cent (5 per cent on average) with up to 34 per cent (18 per cent on average) reduction in peak power dynamic range.

High-level synthesis

High-level synthesis (HLS) tools such as Cadence Stratus and Mentor Catapult help to generate hardware for a given application. However,

unlike bespoke processor design, HLS involves additional development cost since a new high-level specification of application behaviour needs to be defined and the high-level specification itself needs to be verified. Besides, while HLS tools can transform many C programs into efficient ASICs, there are well-known limitations that further increase development costs.

Cadence Stratus. Cadence Stratus HLS delivers up to ten times better productivity than traditional RTL design. It lets you quickly design and verify high-quality RTL implementations from abstract System C, C or C++ models. Using the platform, you can reduce the intellectual property (IP) development cycle from months to weeks.

You can easily create abstract models using Stratus HLS' integrated design environment (IDE) and synthesise optimised hardware from those models. You can then retarget these models to new technology platforms and reuse them more easily than you could with traditional hand-coded RTL. You can actively make tradeoffs between power, area and performance from within the HLS environment. Users have reported productivity as high as two million verified gates/designer/year compared to 200,000 with the traditional RTL flow.

Mentor Catapult. The Catapult HLS platform empowers designers to use industry-standard ANSI C++ and SystemC to describe functional intent and move up to a more productive abstraction level. From these high-level descriptions, Catapult generates production-quality RTL. By speeding time to RTL and by automating the generation of bug-free RTL, it significantly reduces the time to verified RTL.

The Catapult platform pairs synthesis with the power of formal C property checking to find bugs early at the C, C++ or SystemC level and to comprehensively verify source code before synthesis. Its advanced power optimisations automatically provide significant reductions in dynamic power consumption. The highly-interactive Catapult workflow provides full visibility and control of the synthesis process, enabling